
Chapter 3

Machine learning

Olcay Taner Yıldız¹

3.1 Introduction

Today, with the advances in the hardware technologies, it is possible to store, process, and output large amount of data. With the increase in the size of data, explaining it, that is, extracting meaningful information from it, becomes a bottleneck. Machine learning, i.e., the science of extracting useful information from data comes as an aid. Empowered with concepts from mathematics, statistics, and computer science, machine learning is arguably the solution for all of our information extraction problems.

Machine learning is about *curve fitting*. In regression problems, we try to fit a linear/nonlinear function to data. For example, in support vector machine regression, current disposition of the data is not enough and we transform our data by a kernel function to a high-dimensional space and fit the curve in that high-dimensional space [1]. In K -class classification problems, we try to fit a linear/nonlinear function to separate classes. For example, in naïve Bayes, we assume that the data are Gaussian-distributed, and the separating function is easily calculated [2].

Machine learning is about *optimization*. In learning via optimization, we define an error function (loss function) on the model and try to optimize it, i.e., try to find the optimal parameters of the model by optimization techniques borrowed from optimization literature. For example, in multivariate linear regression, the error function is the sum of squared errors, we take the partial derivatives of the error function with respect to the weights, equalize the derivatives to zero to get the linear equations, and solve the equations to extract the weights of the model. Yet in neural-network-based classification, the error function is cross-entropy, we take the partial derivatives of the error function with respect to the weights of the network to get the update equations, and using those update equations we train the network [3].

Machine learning is about *algorithms*. In decision/regression trees, we need to write a recursive-learning algorithm to classify the data arriving at each decision node, and we also need to write a recursive code to generate the decision tree structure [4]. In Bayesian networks, we use graph algorithms to calculate conditional independences between variables, or to calculate marginal probabilities [5,6].

¹Department of Computer Engineering, Işık University, Istanbul, Turkey

Yet in hidden Markov models, we use Viterbi algorithm, a clever application of dynamic programming, to calculate the most probable state sequence given an observation sequence [7].

Machine learning is about *statistics*. We usually assume Gaussian noise on the data, we assume multivariate normal distribution on class covariance matrices in quadratic discriminant analysis, and we use cross-validation or bootstrapping to generate multiple training sets. We also use hypothesis testing to decide which classifier is better than the other based on their classification performance on several test sets [8,9].

Machine learning is about *models*. In decision trees, the data structure is a binary/ L -ary tree depending on the type of the features and the decision function used [10]. In rule learning, the model is an ordered set of rules [11]. In Bayesian networks, hidden Markov models, and neural networks, the most complex model, i.e., graphs are used. In naïve Bayes, linear discriminant analysis (LDA), we use a multivariate polynomial to represent the data.

Machine learning is about *performance metrics*. In classification, we use accuracy if we want to get a crude estimate on the performance of the classifier. If we need more details on pairwise classes, confusion matrix comes in handy. If the dataset has two classes, more metrics follow: precision, recall, true positive rate, false positive rate, F -measure, etc. [12]. In regression, we use mean square error for a general estimate on the performance of the regressor. There are also other performance metrics, such as hinge-loss for specific algorithms, i.e., support vector machines [13].

So what is machine learning about? Like defining an elephant, we need the combination of all of these topics to define machine learning: we start machine learning by assuming a certain *model* on the data, use *algorithmic* and/or *optimization* and/or *statistical* techniques to learn the parameters of that model (which is sometimes defined as *curve fitting*), and use *performance metrics* to evaluate our model/algorithm/classifier.

This chapter is organized as follows: everything begins with data. We will talk about data representation in Section 3.2. Dimension reduction techniques are reviewed in Section 3.3. We discuss basic clustering algorithms in Section 3.4. Classification and regression algorithms are discussed in Section 3.5. Lastly, we conclude with performance assessment of algorithms in Section 3.6.

3.2 Data

In any machine-learning application, we start with data, i.e., measurements/calculations/observations made by the user. The multivariate data are represented by a 2D matrix as:

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_d^{(1)} & y^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_d^{(2)} & y^{(2)} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ x_1^{(N)} & x_2^{(N)} & x_3^{(N)} & \dots & x_d^{(N)} & y^{(N)} \end{bmatrix}$$

where d represents the number of features (variables, attributes, and inputs) and N represents the number of instances (observations and examples) in the dataset. The instances are assumed independent and identically distributed. Each row in the matrix corresponds to a single instance, whereas each column in the matrix corresponds to a single feature. The variable y represents the output feature and (i) takes discrete values for classification and (ii) continuous values for regression problems. Datasets are usually grouped into three: sparse datasets, where $d \gg N$ (i.e., genome datasets, image datasets); *big datasets*, where $N \gg d$ (i.e., text datasets and geo datasets); and standard datasets.

Each feature can be continuous (an integer, a real number) or discrete (taking L distinct values) depending on the application. As an example, let say we have a dataset about $N = 1,000$ students.

$$\mathbf{X} = \begin{bmatrix} 1976 & \text{male} & 2.34 & \text{amber} \\ 1984 & \text{female} & 3.12 & \text{brown} \\ \dots & & & \\ 2000 & \text{female} & 1.23 & \text{blue} \end{bmatrix}$$

where we have four input features, namely, birth year of the student (continuous), sex of the student (discrete with $L = 2$), GPA of the student (continuous), and eye color of the student (discrete with $L = 4$). See Figure 3.1 for the famous Iris dataset plotted with respect to third and fourth features.

Many machine-learning algorithms cannot process discrete features; nevertheless, discrete features can be converted to continuous features via 1-of- L encoding.

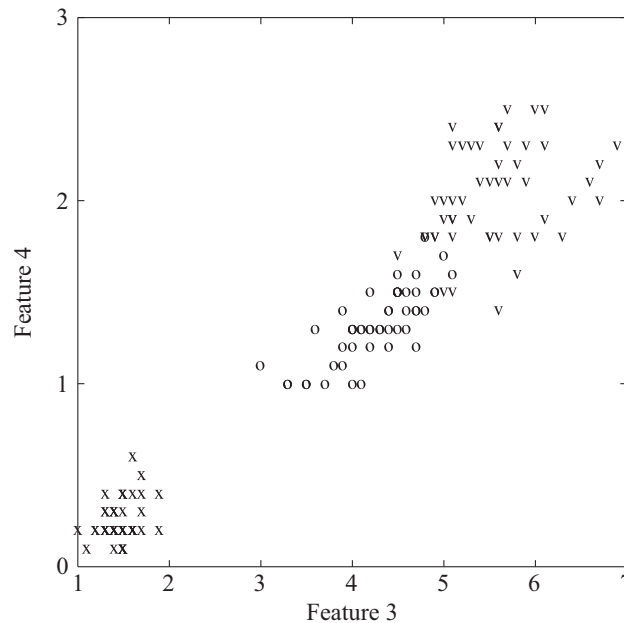


Figure 3.1 Iris dataset

In 1-of- L encoding, each discrete feature is encoded by an extra L feature, where only one of the new features is 1 (corresponding to the value of that feature), and the remaining are 0. As extra features are added, the size of the dataset grows and application of some algorithms may get impractical. For instance, if we convert the previous dataset, we get the following matrix:

$$\mathbf{X} = \begin{bmatrix} 1976 & 1 & 0 & 2.34 & 1 & 0 & 0 & 0 \\ 1984 & 0 & 1 & 3.12 & 0 & 1 & 0 & 0 \\ \dots & & & & & & & \\ 2000 & 0 & 1 & 1.23 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Yet other machine-learning algorithms require abstraction of continuous data, whereby estimation of mean vector:

$$\mathbf{m} = [m_1 m_2 \dots m_d]$$

and covariance matrix:

$$\mathbf{S} = \begin{bmatrix} s_1^2 & s_{12} & \dots & s_{1d} \\ s_{21} & s_2^2 & \dots & s_{2d} \\ \dots & & & \\ s_{d1} & s_{d2} & \dots & s_d^2 \end{bmatrix}$$

is required. In the covariance matrix, the diagonal terms are the variances and the off-diagonal terms are the covariances. If the data are normally distributed, we write $\mathbf{X} \sim \mathcal{N}(\mathbf{m}, \mathbf{S})$.

3.3 Dimension reduction

As a preprocessing step before classification or regression, we may introduce a dimension reduction step, whereby we reduce the number of dimensions of the dataset. Some of the reasons for dimension reduction are as follows: (i) in many learning algorithms the complexity of the algorithm depends on d , and if we reduce d , the training complexity of our algorithm decreases; (ii) ‘‘Occam’s Razor’’ principle: if we can explain the same data with a *model* having fewer parameters, we must choose that model; and (iii) if some input features can be removed, they are unnecessary, so we reduce the cost of extracting those features.

Dimension reduction algorithms can be grouped into two. *Feature-selection* algorithms, where we select a subset of the original features (k features out of d features) [14,15]; *feature-extraction* algorithms, where we generate a new set of k features which are some linear/nonlinear combination of the original features.

3.3.1 Feature selection

In feature selection, in our case subset selection, our objective is to find the best subset of the original features, where the best is determined by the *performance metric*. Since there are 2^d possible feature subsets, exhaustive search is impossible

and we use ourselves into ingenious search algorithms. Depending on the path on the number of selected features k , subset selection algorithms are categorized into three types: forward, backward, and floating, where k increases, decreases, and sometimes increases sometimes decreases, respectively. All three algorithms are actually state space search algorithms, where a state is a subset of features, and search corresponds to generating a new subset from the current best subset.

In forward selection, we start by empty feature subset $F = \emptyset$. At each step, we produce a candidate subset list FL , which includes the subsets obtained by adding one possible feature to F ($FL = \{1\}, \{2\}, \dots, \{d\}$). The current best subset F is selected from the candidate subset list according to the performance metric. The algorithm continues by selecting a new F at each step and terminates when the performance metric does not improve.

In backward selection, we start by the original feature subset $F = \{1, 2, \dots, d\}$. At each step, we again produce a candidate subset list, which includes the subsets obtained by removing one possible feature from F ($FL = \{2, 3, \dots, d\}, \{1, 3, \dots, d\}, \{1, 2, \dots, d-1\}$). The current best subset F is selected from the candidate subset list according to the performance metric. The algorithm continues by selecting a new F at each step and terminates when the performance metric does not improve.

Floating selection is the combination of the previous two algorithms. At each step, when producing SL , we take into account of both adding and removing features [16]. Floating selection can start both from empty and original feature subset.

The best feature-selection algorithm depends on the number of features k in the optimal feature subset. If k is near to 0, forward selection is more hopeful to find the optimal subset, whereas if k is near to d , backward can be lucky. Although time complexity of floating selection is more than the other two, it could be very helpful if we do not know k .

3.3.2 Feature extraction

In feature extraction, we generate a totally new set of features from the original feature set. The feature-extraction algorithms map the data to a new space, where the mapping can be linear or nonlinear, and the mapping can be supervised or unsupervised (will use output y or will not). In this chapter, we will talk about principal component analysis and LDA as feature-extraction algorithms, where the aim is to find the projection matrix $\mathbf{W}$ to get the new dataset via the following equation:

$$\mathbf{X} = \mathbf{X}\mathbf{W} \quad (3.1)$$

Principal component analysis (PCA) is an unsupervised linear feature-extraction algorithm, which maximizes the variance throughout the projected features, where each projection vector $\mathbf{w}_i$ is orthogonal to other projection vectors [2]. So, the problem reduces into an *optimization* problem:

$$\begin{aligned} & \underset{\mathbf{W}}{\text{maximize}} && \mathbf{W}\mathbf{S}\mathbf{W}^T \\ & \text{subject to} && \|\mathbf{w}_i\| = 1, \quad i = 1, \dots, d, \\ & && \mathbf{w}_i^T \mathbf{w}_j = 0, \quad i, j = 1, \dots, d; \quad i \neq j, \end{aligned}$$

whereby the eigenvectors of the covariance matrix $\mathbf{S}$ are the solution to the optimization problem. The eigenvectors of $\mathbf{S}$ are sorted according to their corresponding eigenvalues λ and the most important k eigenvectors are the ones having the largest k eigenvalues. Figure 3.2(a) shows the application of PCA on *Iris* dataset. We can also control the dimension reduction (value of k) according to the proportion of variance explained.

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

LDA is a supervised linear feature-extraction algorithm which separates instances from different classes as much as possible and brings together instances from the same class as much as possible [2]. Between-class scatter matrix $\mathbf{S}_B = \sum_{i=1}^K N_i(\mathbf{m}_i - \mathbf{m})(\mathbf{m}_i - \mathbf{m})^T$ and within-class scatter matrix $\mathbf{S}_W = \sum_{i=1}^K \mathbf{S}_i$ of a K -class dataset quantifies the separability between and within classes, respectively. If we write the problem as an *optimization* problem:

$$\begin{aligned} & \underset{\mathbf{W}}{\text{maximize}} && \frac{|\mathbf{W}\mathbf{S}_B\mathbf{W}^T|}{|\mathbf{W}\mathbf{S}_W\mathbf{W}^T|} \\ & \text{subject to} && \|\mathbf{w}_i\| = 1, \quad i = 1, \dots, d, \\ & && \mathbf{w}_i^T \mathbf{w}_j = 0, i, j = 1, \dots, d; i \neq j \end{aligned}$$

then the eigenvectors of the matrix $\mathbf{S}_W^{-1} \mathbf{S}_B$ will be the solution to this optimization problem. In LDA, the maximum rank of $\mathbf{S}_B$ is $K - 1$, therefore $k \leq K - 1$. Figure 3.2(b) shows the application of LDA on *Iris* dataset.

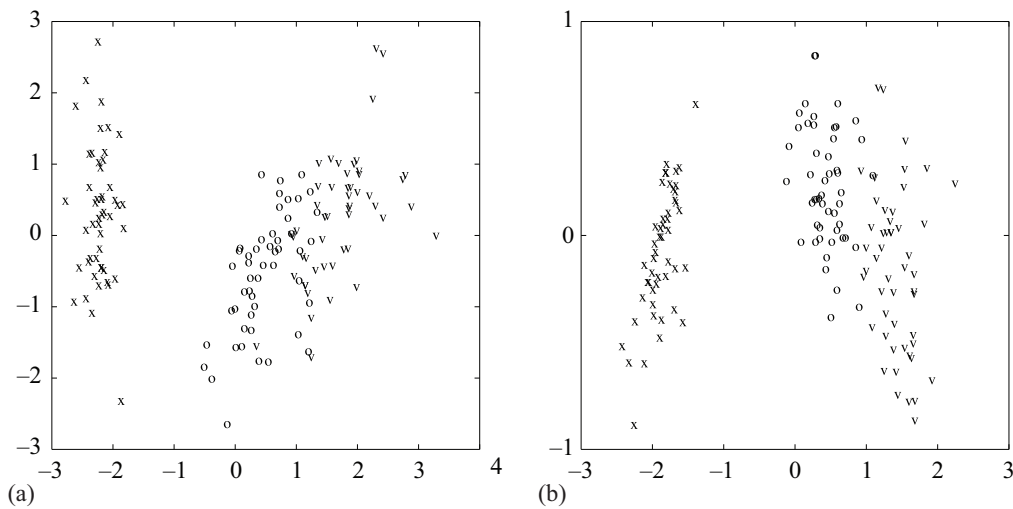


Figure 3.2 Feature extraction on *Iris* dataset: (a) PCA and (b) LDA

3.4 Clustering

Before delving into the supervised algorithms, we will briefly introduce one of the most important topics in unsupervised machine learning, namely, clustering. In clustering, the aim is to divide $\mathbf{X}$ into a set of groups (clusters, sets), where the groups are disjoint and possibly represent meaningful divisions. After clustering, the groups can be named by experts in the respective domains, which further simplify the descriptive *model* of the data. In this chapter, we will talk about K -means clustering and hierarchical clustering [17,18].

3.4.1 K -Means clustering

In K -means clustering, we are interested in splitting the data $\mathbf{X}$ into K disjoint groups $\mathbf{X}_i$, where the group means $\mathbf{m}_i$ form the representers of the groups. In this problem, there are two sets of unknowns. The first set of unknowns is the elements of the group membership matrix $\mathbf{G}$, where $g_i^{(t)}$ is 1, if the instance $\mathbf{x}^{(t)}$ belongs to the group i , 0 otherwise. The second set of unknowns is group means $\mathbf{m}_i$. The important and interesting thing is, these two sets of unknowns are bound to each other. If we know one set of unknowns, we can optimally determine the other set of unknowns based on the optimization of the reconstruction error.

$$E(\mathbf{m}_i, \mathbf{G}|\mathbf{X}) = \sum_t \sum_i g_i^{(t)} \|\mathbf{x}^{(t)} - \mathbf{m}_i\|^2 \quad (3.2)$$

If we know $\mathbf{G}$, we can calculate group means as:

$$\mathbf{m}_i = \frac{\sum_t g_i^{(t)} \mathbf{x}^{(t)}}{\sum_t g_i^{(t)}} \quad (3.3)$$

On the other hand, if we know $\mathbf{m}_i$, we can extract the group membership of $\mathbf{x}^{(t)}$, because the Euclidean distance of an instance $\mathbf{x}^{(t)}$ to the group means determines the group membership as:

$$g_i^{(t)} = \begin{cases} 1 & \text{if } \|\mathbf{x}^{(t)} - \mathbf{m}_i\| = \min_j \|\mathbf{x}^{(t)} - \mathbf{m}_j\| \\ 0 & \text{otherwise} \end{cases} \quad (3.4)$$

To solve for these two sets of unknowns, we will use a special version of expectation maximization optimization technique and proceed with an iterative procedure. First, we initialize the mean vectors $\mathbf{m}_i$ randomly. Then, at each iteration, we first use (3.4) to calculate the group membership matrix $\mathbf{G}$. Then, once we have calculated $\mathbf{G}$, we can easily calculate group means using (3.3). These two steps are repeated (calculate $\mathbf{G} \rightarrow$ calculate $\mathbf{m}_i \rightarrow$ calculate $\mathbf{G} \rightarrow$ calculate $\mathbf{m}_i \rightarrow$ etc.) until the group membership matrix $\mathbf{G}$ stabilizes.

The most important disadvantage of this iterative procedure is its initialization. Since the optimization technique is a local search procedure, final group means are heavily dependent on the initial group means. Possible techniques, such as

randomly selecting K instances as initial mean vectors, are proposed in the literature to overcome this drawback [2].

K -means clustering can be generalized to soft clustering by:

- Assigning a probability value to $g_i^{(t)}$. Again $\sum_i g_i^{(t)} = 1$, but this way an instance $\mathbf{x}^{(t)}$ will belong to more than one cluster, and the cluster membership decision will be soft (a probability value between 0 and 1) instead of hard (only 0 and 1).
- Assuming the groups that are Gaussian distributed with group covariances $\mathbf{S}_i$, i.e., $\mathbf{X}_i \sim \mathcal{N}(\mathbf{m}_i, \mathbf{S}_i)$.
- Optimizing the log-likelihood instead of the reconstruction error. The expectation maximization procedure appears here again [19,20], and with a similar iterative procedure, we optimize the parameters $\mathbf{m}_i$, $\mathbf{S}_i$, and $\mathbf{G}$.

3.4.2 Hierarchical clustering

Instead of clustering instances with respect to the cluster means, we can cluster instances with respect to their similarities to other instances. In other words, instance to instance distance matters, but not the distance of instance to cluster mean. The most distant the instances, the more dissimilar they are.

Hierarchical clustering is a type of agglomerative clustering, where we start with N clusters (where each instance is actually a cluster) and merge clusters one by one until only one cluster remains. At each iteration, we select the two closest clusters to merge and get a single merged cluster. But how will we decide on the closest clusters, or in other words, how will we compute the distance between clusters, where until now we have only talked about distances between instances.

In single-link clustering, the distance between two clusters is the smallest distance between all possible pairs of instances of the two groups. So, single-link clustering has an optimistic point of view, if there is only a single pair of instances which are very near, then single-link clustering thinks these two clusters as very close. In complete-link clustering, the distance between two clusters is the largest distance between all possible pairs of instances of the two groups. Yet, complete-link clustering has a pessimistic point of view, if there is only a single pair of instances which are very far, then complete-link clustering expects these two clusters as very distant.

In average-link clustering, the distance between two clusters is the average of all distances between all possible pairs of instances of the two groups. In centroid-link clustering, the distance between two clusters is the distance between the cluster means.

3.5 Supervised learning algorithms

In this section, we will briefly review supervised learning algorithms used in machine learning. Supervised algorithms can be categorized in multiple ways. First, supervised algorithms can be parametric vs nonparametric. In the parametric algorithms, the algorithm makes certain assumptions on the underlying distribution of the data (usually called *bias*) and fits the model based on those assumptions.

Nevertheless, in the nonparametric case, the situation is reverse. The algorithm does not make any assumption on the data, and the model is actually the data itself.

Second, supervised algorithms can be discriminative vs generative. In the discriminative algorithms, the algorithm's aim is to find such a discriminant that it will separate the classes as best as it gets. Therefore, only the instances near the discriminant are important. On the other hand, generative algorithms make certain distribution assumptions on the data and use that assumption to produce a discriminator. For this case, since all data are required for the satisfaction of the assumption, all instances are equally important.

Third, supervised algorithms can be classification vs regression. In classification, the output feature $\mathbf{y}$ takes discrete values. In regression, the output feature $\mathbf{y}$ takes continuous values. For classification, only counting the mismatches in the output is enough, therefore the loss function is accuracy; whereas for regression, there is actually no mismatch but a real valued absolute discrepancy in the output value, therefore the loss function is squared error.

3.5.1 Simplest algorithm: prior

Before introducing more meaningful supervised algorithms, we must first talk about the simplest algorithm ever, the base case, or the so-called prior. According to Occam's Razor' principle, if we can explain the same data with a *model* having fewer parameters, we must choose that model. Leading that principle, the simplest model about the data is the prior distribution of the data. For classification, prior distribution corresponds to the prior probabilities of the classes, so the prior classifier labels all the test data with the most probable class label; whereas for regression, prior distribution corresponds to the mean value of the output vector $\mathbf{y}$; and the prior regressor assigns the mean value to all test data.

If we propose a new classification/regression algorithm on a specific dataset, we must be ensured that it must beat the prior. Note also that according to the *no free lunch theorem* [21], there does not exist any algorithm that can beat every algorithm on all datasets. So, there can be some datasets, where prior is the superior.

3.5.2 A simple but effective algorithm: nearest neighbor

The most commonly used representative of nonparametric algorithms is the nearest neighbor. The assumption of nearest neighbor is simple, the world does not change much, i.e., similar things perform similarly. Therefore, we only need to store the dataset itself and make the decision on the test instance based on the similarity of it to the instances in the dataset. In other words, the class label (regression value) of an instance is strongly influenced by its *nearby* instances.

More formally, in k -nearest neighbor classification, the class-label of a test instance is obtained by taking the majority vote of the k -nearest instances in the training set [22]. Ties are broken arbitrarily, where k is usually taken as an odd number to get rid of ties mostly. In k -nearest neighbor regression, the output value of a test instance is calculated by taking the average of the output values of the k -nearest examples in the training set. Euclidean distance is usually used to calculate the distance between two instances.

In machine-learning literature, nearest neighbor techniques are also called instance-based or memory-based learning algorithms, because they simply store the training set in a table and look up the output values of the nearest k instances [23]. They are also called lazy learning algorithms, since they do not do anything in the training phase of the algorithm, but start processing while doing testing phase. Although the training complexity of nearest neighbor is as negligible as prior, the test complexity is one of the largest in all learning algorithms ($\mathcal{O}(N)$ for a single test instance). In spite of this drawback, due to its simplicity, k -nearest neighbor is one of the widely used and one of the successful learning algorithms in practice. Theoretically, it has been shown that for larger datasets ($N \rightarrow \infty$), the risk of 1-nearest neighbor is never worse than the best risk that can be achieved [24].

3.5.3 Parametric methods: five shades of complexity

Starting from the Bayes' rule, we have:

$$p\{C_i|\mathbf{x}\} = \frac{p(\mathbf{x}|C_i)p(C_i)}{\sum_{j=1}^K p(\mathbf{x}|C_j)p(C_j)} \quad (3.5)$$

where $p(C_i)$, $p(C_i|\mathbf{x})$, and $p(\mathbf{x}|C_i)$ represent the prior probability, the posterior probability, and the class distribution of the class C_i , respectively. As we can see, if all class distributions are equal, choosing the maximum posterior probability reduces to the choosing the maximum prior probability, i.e., *prior algorithm*.

Since the denominator is the same for all classes, we can simplify the posterior probability and get the discriminant function as $f_i(\mathbf{x}) = \log p(\mathbf{x}|C_i) + \log p(C_i)$. If the class distributions are assumed to follow Gaussian density, we have $p(\mathbf{x}|C_i) \sim \mathcal{N}(\mathbf{m}_i, \mathbf{S}_i)$, and the discriminant function reduces to:

$$f_i(\mathbf{x}) = -\frac{d \log 2\pi}{2} - \frac{\log |\mathbf{S}_i|}{2} - \frac{(\mathbf{x} - \mathbf{m}_i)^T \mathbf{S}_i^{-1} (\mathbf{x} - \mathbf{m}_i)}{2} + \log p(C_i) \quad (3.6)$$

The first part is the same for all classes, and if we get rid of it, we obtain our first parametric classifier, namely, *quadratic discriminant*.

$$f_i(\mathbf{x}) = -\frac{\log |\mathbf{S}_i| + \mathbf{x}^T \mathbf{S}_i^{-1} \mathbf{x} - 2\mathbf{x}^T \mathbf{S}_i^{-1} \mathbf{m}_i + \mathbf{m}_i^T \mathbf{S}_i^{-1} \mathbf{m}_i}{2} + \log p(C_i) \quad (3.7)$$

The number of parameters, i.e., the model complexity is $Kd + Kd(d+1)/2$, where the first part is for class means and the second part for class covariance matrices.

For complexity reduction, we can assume a single shared covariance matrix for all classes. In this case, first and second parts of the equation are the same for all classes and simplifying it reduces $f_i(\mathbf{x})$ to our second classifier, namely, *linear discriminant*.

$$f_i(\mathbf{x}) = \mathbf{x}^T \mathbf{S}_i^{-1} \mathbf{m}_i - \frac{\mathbf{m}_i^T \mathbf{S}_i^{-1} \mathbf{m}_i}{2} + \log p(C_i) \quad (3.8)$$

The model complexity is $Kd + d(d + 1)/2$, where the first part is for class means and the second part for shared covariance matrix.

For more reduction, we assume all off-diagonals of the shared covariance matrix are zero. For this case, first and second parts of the equation simplify to vector multiplications instead of matrix multiplications, and we get the *naïve Bayes* classifier. The model complexity is $Kd + d$, where the first part is for class means and the second part for the diagonal of the shared covariance matrix.

As the fifth algorithm, we further reduce by taking priors equal and a single covariance value s . In this case, we get the *nearest mean* classifier and the model complexity is only $Kd + 1$.

Depending on the application, we can choose our complexity level (as bias) and report the test error. Yet, another possibility may well be applying all algorithms and make a selection considering both the test error and model complexity [25].

3.5.4 Decision trees

Decision trees are significantly different than the previous models we have discussed. First, they have a tree-based structure where each non-leaf node m implements a decision function, $f_m(\mathbf{x})$, and each leaf node corresponds to a class decision. Second, they are one of most interpretable learning algorithms available. When written as a set of IF-THEN rules, the decision tree can be transformed into a human-readable format, which then can be modified and/or validated by human experts in their corresponding domains.

Decision trees are categorized depending on the type of the decision function. In univariate decision trees [4] (Figure 3.3(a)), the decision function $f_m(\mathbf{x})$ uses only one feature x_i and depending on the type of that feature, each non-leaf node of a univariate decision tree will have two or L children. If x_i is a continuous feature, the decision function is in the form of $x_i < \theta$, and each non-leaf node has two children, where the instances satisfying (not satisfying) the decision function follow the left (right) child, respectively. If x_i is a discrete feature, the decision function takes on

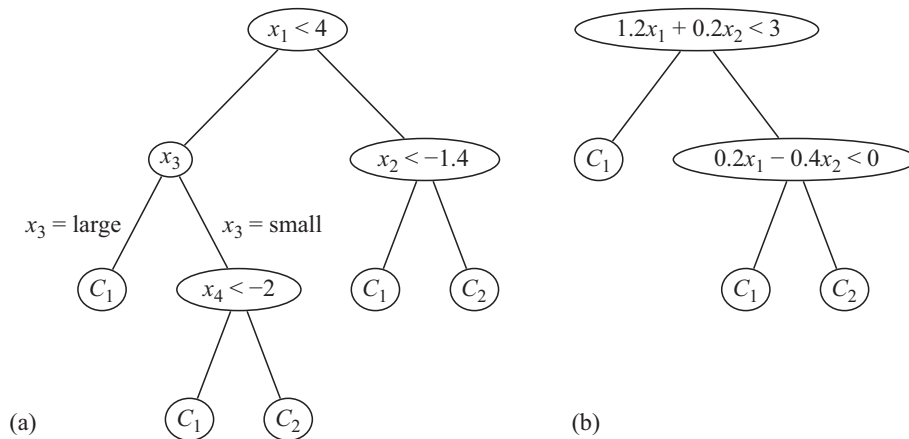


Figure 3.3 Decision trees: (a) univariate and (b) multivariate

the forms $x_i = v_1, x_i = v_2, \dots, x_i = v_L$, where $v_1, v_2, \dots, v_L$ correspond to all possible values of the discrete feature x_i .

In multivariate decision trees (Figure 3.3(b)), the decision function $f_m(\mathbf{x})$ uses all features of $\mathbf{x}$ and each non-leaf node has two children. Depending on the type of the multivariate function, we have (i) multivariate linear tree [26,27], where the decision function is in the form of $\mathbf{w}^T \mathbf{x} < \theta$, (ii) multivariate nonlinear tree [28], where the decision function is a nonlinear function in terms of $\mathbf{x}$, and (iii) omnivariate tree [10], where the decision function can be any of the previous three.

Tree induction algorithms are recursive and at each decision node m , starting from the root node (we have the complete training set), we look for the best decision function $f_m(\mathbf{x})$. When the best decision function is found, the training data are split according to $f_m(\mathbf{x})$, and the learning continues recursively with the children of m . We continue splitting until there is no need for the same, where the instances are all from the same class.

For decision trees, the *loss function*, i.e., the criterion for comparing the candidate decision functions is quantified by an impurity measure. Several impurity measures are proposed in the literature such as Gini index [29], entropy.

$$\text{Entropy} = \sum_{i=1}^K p(C_i) \log p(C_i) \quad (3.9)$$

where $p(C_i)$ is the probability of class C_i at node m [30].

3.5.5 Kernel machines

Kernel machines, in other words, support vector machines [1,31], are maximum margin methods, where the model is written as a weighted sum of *support vectors*. Kernel machines are discriminative methods, i.e., they are only interested in the instances across the class boundaries in classification, or instances across the regressor in regression. For obtaining the optimal separating hyperplane, kernel machines try to maximize separability, or *margin*, and write the problem as a quadratic optimization problem, whose solution gives us support vectors.

Kernel functions improve our notion of instance model, and not only we can define kernel functions on instances but we can also define on networks, graphs, words, sentences, or trees. This great range of applicability of kernel functions made kernel functions popular across many domains, including natural language processing, bioinformatics [32], and robotics.

3.5.5.1 Separable case: optimal separating hyperplane

For a linearly separable two-class problem (see Figure 3.4(a)), we define the output $y^{(t)}$ as $+1/-1$ and the aim is to find the optimal separating hyperplane $\mathbf{w}$ and bias w_0 which satisfies the following condition:

$$y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) \geq 1 \quad (3.10)$$

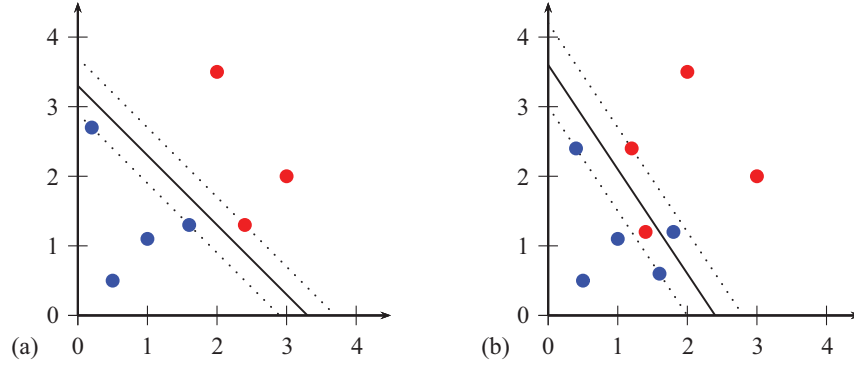


Figure 3.4 For a two-class problem the separating hyperplane and support vectors: (a) separable case and (b) nonseparable case

For the instances of positive class ($y^{(t)} = +1$), we want to see them on the positive side of the hyperplane. Similarly, for the instances of negative class ($y^{(t)} = -1$), we want to see them on the negative side of the hyperplane. Note that we also want the instances some distance away from the hyperplane, which we call *margin*, which we also want to maximize for the best generalization. If we write the margin maximization problem as an optimization problem:

$$\begin{aligned} & \underset{\mathbf{w}}{\text{minimize}} && \frac{\|\mathbf{w}\|^2}{2} \\ & \text{subject to} && y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) \geq 1, \quad t = 1, \dots, N \end{aligned} \quad (3.11)$$

This standard quadratic optimization problem can be converted to its dual formulation using Lagrange multipliers $\alpha^{(t)}$ as:

$$L_p = \frac{\|\mathbf{w}\|^2}{2} - \sum_t \alpha^{(t)} [y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) - 1] \quad (3.12)$$

With Karush–Kuhn–Tucker conditions $\partial L_p / \partial \mathbf{w} = 0$ and $\partial L_p / \partial w_0 = 0$, we get the following dual formulation:

$$\begin{aligned} & \underset{\alpha^{(t)}}{\text{maximize}} && \sum_t \alpha^{(t)} - \frac{\sum_t \sum_s \alpha^{(t)} \alpha^{(s)} y^{(t)} y^{(s)} (\mathbf{x}^{(t)})^T \mathbf{x}^{(s)}}{2} \\ & \text{subject to} && \sum_t \alpha^{(t)} y^{(t)} = 0 \\ & && \alpha^{(t)} \geq 0, \quad t = 1, \dots, N. \end{aligned} \quad (3.13)$$

Once we solve the previous dual quadratic optimization problem, we obtain a set of instances for which $\alpha^{(t)} > 0$. These instances are called support vectors and they lie on the margin.

3.5.5.2 Nonseparable case: soft margin hyperplane

For a linearly nonseparable two-class problem (see Figure 3.4(b)), we look for the hyperplane which acquires the minimum error. Since some of the constraints will not be satisfied, we add slack variables $\xi^{(t)} \geq 0$, which store the deviation from the margin. Now the aim is to find the optimal $\mathbf{w}$ and bias w_0 which satisfies the following condition:

$$y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) \geq 1 - \xi^{(t)} \quad (3.14)$$

Since each slack variable contributes to error, the total error is $\sum_t \xi^{(t)}$ and including this error as a weighted penalty in the objective function transforms the margin maximization problem to:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \frac{\|\mathbf{w}\|^2}{2} + C \sum_t \xi^{(t)} \\ \text{subject to} \quad & y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) \geq 1 - \xi^{(t)}, \quad t = 1, \dots, N. \\ & \xi^{(t)} \geq 0, \quad t = 1, \dots, N \end{aligned} \quad (3.15)$$

Again, we can convert this problem into its dual by introducing Lagrange multipliers $\alpha^{(t)}$ and $\beta^{(t)}$, and obtain:

$$L_p = \frac{\|\mathbf{w}\|^2}{2} + C \sum_t \xi^{(t)} - \sum_t \alpha^{(t)} [y^{(t)}(\mathbf{w}^T \mathbf{x}^{(t)} + w_0) - 1 + \xi^{(t)}] - \sum_t \beta^{(t)} \xi^{(t)} \quad (3.16)$$

With Karush–Kuhn–Tucker conditions solved, we get the following dual formulation:

$$\begin{aligned} \underset{\alpha^{(t)}}{\text{maximize}} \quad & \sum_t \alpha^{(t)} - \frac{\sum_t \sum_s \alpha^{(t)} \alpha^{(s)} y^{(t)} y^{(s)} (\mathbf{x}^{(t)})^T \mathbf{x}^{(s)}}{2} \\ \text{subject to} \quad & \sum_t \alpha^{(t)} y^{(t)} = 0 \\ & 0 \leq \alpha^{(t)} \leq C, \quad t = 1, \dots, N \end{aligned} \quad (3.17)$$

Again, once we solve the quadratic problem, the instances having the corresponding value of $\alpha^{(t)} > 0$ are support vectors.

C is the regularization hyperparameter of the support vector machines and is usually tuned between 2^{-10} and 2^{10} with multipliers of 2. The upper bound for the training time complexity is $\mathcal{O}(N^3)$.

When there are more than two classes ($K > 2$), we resort to multi class to binary class conversion techniques. In one-vs-all strategy, we define K subproblems, and in each subproblem, we try to separate class i from all other classes. On the other hand, in one-vs-one strategy, we define $K(K-1)/2$ subproblems, and in each subproblem, we try to separate class i from class j .

3.5.5.3 Kernel trick

If we had stopped here, support vector machines would not have been so popular. In order to solve a nonlinear problem, we do not fit a nonlinear model but map the original input space into a high-dimensional space and solve the problem linearly in that space. So instead of linearly combining the instances, we linearly combine the projected instances.

Again, the aim is to find the optimal $\mathbf{w}$ which satisfies the following condition:

$$y^{(t)} \mathbf{w}^T \Phi(\mathbf{x}^{(t)}) \geq 1 - \xi^{(t)} \quad (3.18)$$

where $\Phi(\mathbf{x}^{(t)})$ projects the instance $\mathbf{x}^{(t)}$ into a high-dimensional space. Including the error as a weighted penalty in the objective function transforms the margin maximization problem to:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \frac{\|\mathbf{w}\|^2}{2} + C \sum_t \xi^{(t)} \\ \text{subject to} \quad & y^{(t)} \mathbf{w}^T \Phi(\mathbf{x}^{(t)}) \geq 1 - \xi^{(t)}, \quad t = 1, \dots, N, \\ & \xi^{(t)} \geq 0, \quad t = 1, \dots, N. \end{aligned} \quad (3.19)$$

Yet again we can convert this problem into its dual by introducing Lagrange multipliers $\alpha^{(t)}$ and $\beta^{(t)}$, and obtain:

$$L_p = \frac{\|\mathbf{w}\|^2}{2} + C \sum_t \xi^{(t)} - \sum_t \alpha^{(t)} [y^{(t)} \mathbf{w}^T \Phi(\mathbf{x}^{(t)}) - 1 + \xi^{(t)}] - \sum_t \beta^{(t)} \xi^{(t)}. \quad (3.20)$$

With Karush–Kuhn–Tucker conditions solved, and replacing the inner product of $\Phi(\mathbf{x}^{(t)})^T \Phi(\mathbf{x}^{(s)})$ with kernel function $K(\mathbf{x}^{(t)}, \mathbf{x}^{(s)})$, we get the following dual formulation:

$$\begin{aligned} \underset{\alpha^{(t)}}{\text{maximize}} \quad & \sum_t \alpha^{(t)} - \frac{\sum_t \sum_s \alpha^{(t)} \alpha^{(s)} y^{(t)} y^{(s)} K(\mathbf{x}^{(t)}, \mathbf{x}^{(s)})}{2} \\ \text{subject to} \quad & \sum_t \alpha^{(t)} y^{(t)} = 0 \\ & 0 \leq \alpha^{(t)} \leq C, \quad t = 1, \dots, N. \end{aligned} \quad (3.21)$$

So instead of mapping two instances to a high-dimensional space and applying dot product there, we can apply the kernel function directly in the original space. This is the main idea behind the *kernel trick*. Possible kernel functions used in the literature are:

- Polynomial kernel of degree n : $K(\mathbf{x}^{(t)}, \mathbf{x}^{(s)}) = (\mathbf{x}^{(t)} \mathbf{x}^{(s)} + 1)^n$
- Radial basis function: $K(\mathbf{x}^{(t)}, \mathbf{x}^{(s)}) = \exp\left(-\frac{\|\mathbf{x}^{(t)} - \mathbf{x}^{(s)}\|^2}{2s^2}\right)$
- Sigmoidal function: $K(\mathbf{x}^{(t)}, \mathbf{x}^{(s)}) = \tanh(2\mathbf{x}^{(t)} \mathbf{x}^{(s)} + 1)$

3.5.6 Neural networks

Artificial neural networks (ANN) take their inspiration from the brain. The brain consists of billions of neurons and these neurons are interconnected and work in parallel, which makes the brain a powerful computing machine. Each neuron is connected through synapses to thousands of neurons and the *firing* of a neuron depends on those synapses. Research on ANN started with the invention of perceptron [33] but came to its first halt with its limitation on the XOR problem [34]. After 15 years of standby, the resurrection of ANN came with the paper [35].

3.5.6.1 Neurons (units)

There are three types of neurons (units) in ANN. Each unit except the input unit takes an input and calculates an output. Input units represent a single input feature x_i or the bias $+1$.

Hidden units calculate an intermediate output from its inputs. They first combine their inputs linearly as $\mathbf{w}^T \mathbf{x}$ and then use nonlinear activation functions to map that linear combination to a nonlinear space. Well-known activation functions are sigmoid $z(x) = 1/(1 + e^{-x})$, hyperbolic tangent $z(x) = (e^x - e^{-x})/(e^x + e^{-x})$, inverse of tangent $z(x) = \tan^{-1}(x)$, and rectified linear unit $z(x) = \max(0, x)$.

Output units calculate the output of the ANN. For regression problems, the output unit only calculates linear combination of its inputs (see Figure 3.5(a) and (c)). For two-class classification problems, the output unit uses sigmoid function $1/(1 + e^{-x})$ to map the output to a probability value (see Figure 3.5(a) and (c)). For K -class classification problems (see Figure 3.5(b) and (d)), there are K output units, and each output unit i uses softmax function $e^{o_i} / \sum_{j=1}^K e^{o_j}$ to map its output to a probability value, so that the sum of all output units is 1.

3.5.6.2 Models and forward-propagation

In this chapter, we are interested in four types of neural networks models (see Figure 3.5), (i) perceptron for regression and two-class classification, (ii) perceptron for K -class classification, (iii) multilayer perceptron for regression and two-class classification, and (iv) multilayer perceptron for K -class classification.

In linear perceptron or simply perceptron, there are two layers, (i) input layer consisting of $d + 1$ input units (including bias unit) and (ii) output layer consisting of one output for regression (K outputs for K -class classification). The weights are represented by $\mathbf{W}$, where w_{ij} represents the weight of the connection between the output unit i and input unit j .

Forward-propagation in linear perceptron calculates the values of output units. For regression, the output is $o = \sum_{j=0}^d w_{0j}x_j$. For two-class classification, the output is $o = 1/(1 + e^{-z})$, where $z = \sum_{j=0}^d w_{0j}x_j$. For K -class classification, the output unit i is $o_i = e^{z_i} / \sum_{k=1}^K e^{z_k}$ where $z_i = \sum_{j=0}^d w_{ij}x_j$. In multilayer perceptron, there are three layers, (i) input layer consisting of $d + 1$ input units (including bias unit), (ii) hidden layer consisting of H hidden units and one bias unit, and (iii) output layer consisting of one output for regression (K outputs for K -class classification).

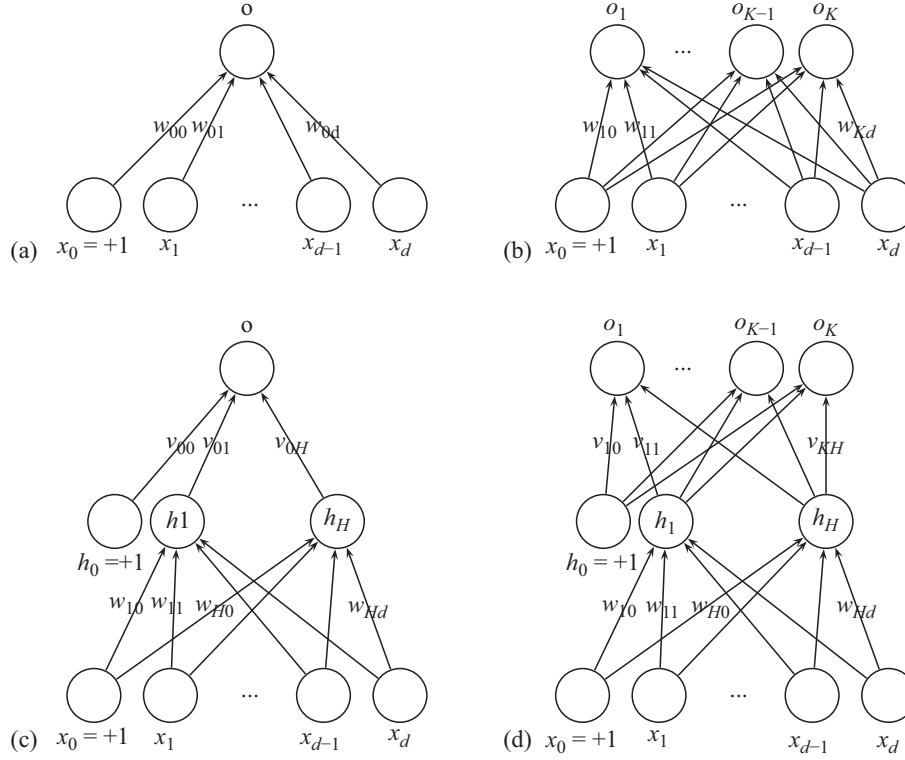


Figure 3.5 Neural network models: (a) perceptron for regression and two-class classification, (b) perceptron for K -class classification, (c) multilayer perceptron for regression and two-class classification, (d) multilayer perceptron for K -class classification

The weights between input and hidden units are represented by $\mathbf{W}$, where w_{ij} represents the weight of the connection between the hidden unit i and input unit j . On the other hand, the weights between hidden units and output units are represented by $\mathbf{V}$, where v_{ij} represents the weight of the connection between the output unit i and hidden unit j .

Forward-propagation in multilayer perceptron calculates the values of hidden and output units. For all models, if the activation function is sigmoid, the value of hidden unit i is $h_i = 1/(1 + e^{-g_i})$, where $g_i = \sum_{j=0}^d w_{ij}x_j$. For regression, the output is $o = \sum_{j=0}^H v_{0j}h_j$. For two-class classification, the output is $o = 1/(1 + e^{-z})$, where $z = \sum_{j=0}^H v_{0j}h_j$. For K -class classification, the output unit i is $o_i = e^{z_i} / \sum_{k=1}^K e^{z_k}$, where $z_i = \sum_{j=0}^H v_{ij}h_j$.

3.5.6.3 Backward-propagation

Before delving into learning in ANN, we need to verify two notions; what is to be learned and what is to be optimized. In regression, we optimize the mean square error:

$$E = (y^{(t)} - o^{(t)})^2, \quad (3.22)$$

in two-class classification we optimize the cross-entropy:

$$E = -y^{(t)} \log o^{(t)} - (1 - y^{(t)}) \log(1 - o^{(t)}), \quad (3.23)$$

in K -class classification, again we optimize cross-entropy but calculated on K classes:

$$E = - \sum_{k=0}^K y_k^{(t)} \log o_k^{(t)} \quad (3.24)$$

For all of the networks, the aim is to learn the parameters of the network, which are simply the weight matrix $\mathbf{W}$ (and $\mathbf{V}$ for multilayer perceptron) [3].

Since all the functions in ANN are differentiable and not easily solvable, one resorts to gradient-descent style optimization. In gradient-descent optimization, we take the partial derivatives of to be optimized function with respect to be learned parameters $((\partial E / \partial \mathbf{W}), (\partial E / \partial \mathbf{V}))$ and use those partial derivatives to calculate the update rules for those parameters.

For regression in linear perceptron, the update rule is:

$$\Delta w_{0j} = -\eta \frac{\partial E}{\partial w_{0j}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial w_{0j}} = \eta(y^{(t)} - o^{(t)})x_j. \quad (3.25)$$

For two-class classification in linear perceptron, the update rule is:

$$\Delta w_{0j} = -\eta \frac{\partial E}{\partial w_{0j}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial z} \frac{\partial z}{\partial w_{0j}} = \eta(y^{(t)} - o^{(t)})x_j. \quad (3.26)$$

For K -class classification in linear perceptron, the update rule is:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta \frac{\partial E}{\partial o_i} \frac{\partial o_i}{\partial z_i} \frac{\partial z_i}{\partial w_{ij}} = \eta(y^{(t)} - o^{(t)})x_j. \quad (3.27)$$

For regression in multilayer perceptron, the update rules are:

$$\Delta v_{0j} = -\eta \frac{\partial E}{\partial v_{0j}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial v_{0j}} = \eta(y^{(t)} - o^{(t)})h_j \quad (3.28)$$

$$\begin{aligned} \Delta w_{ij} &= -\eta \frac{\partial E}{\partial w_{ij}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial h_i} \frac{\partial h_i}{\partial g_i} \frac{\partial g_i}{\partial w_{ij}} \\ &= \eta(y^{(t)} - o^{(t)})v_{0j}h_i(1 - h_i)x_j \end{aligned} \quad (3.29)$$

For two-class classification in multilayer perceptron, the update rules are:

$$\Delta v_{0j} = -\eta \frac{\partial E}{\partial v_{0j}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial z} \frac{\partial z}{\partial v_{0j}} = \eta(y^{(t)} - o^{(t)})h_j \quad (3.30)$$

$$\begin{aligned} \Delta w_{ij} &= -\eta \frac{\partial E}{\partial w_{ij}} = -\eta \frac{\partial E}{\partial o} \frac{\partial o}{\partial z} \frac{\partial z}{\partial h_i} \frac{\partial h_i}{\partial g_i} \frac{\partial g_i}{\partial w_{ij}} \\ &= \eta(y^{(t)} - o^{(t)})v_{0j}h_i(1 - h_i)x_j \end{aligned} \quad (3.31)$$

For K -class classification in multilayer perceptron, the update rules are:

$$\Delta v_{ij} = -\eta \frac{\partial E}{\partial v_{ij}} = -\eta \frac{\partial E}{\partial o_i} \frac{\partial o_i}{\partial z_i} \frac{\partial z_i}{\partial v_{ij}} = \eta (y^{(t)} - o^{(t)}) h_j \quad (3.32)$$

$$\begin{aligned} \Delta w_{ij} &= -\eta \frac{\partial E}{\partial w_{ij}} = -\eta \frac{\partial E}{\partial o_k} \frac{\partial o_k}{\partial z_k} \frac{\partial z_k}{\partial h_i} \frac{\partial h_i}{\partial g_i} \frac{\partial g_i}{\partial w_{ij}} \\ &= \eta \left(\sum_{k=0}^K (y_k^{(t)} - o_k^{(t)}) v_{ij} \right) h_i (1 - h_i) x_j \end{aligned} \quad (3.33)$$

3.6 Performance assessment and comparison of algorithms

Machine learning experiments are usually done to assess the performance of learning algorithms. If we have more than one candidate algorithm to experiment, we face with the second objective: comparing the performance of algorithms.

The first thing we need to know is, we cannot compare algorithms based on the training error. Training errors are optimistic and overly biased. The algorithm tries its best to learn the dataset and therefore is highly prone to recognize (remember) instances instead of learning the relationships among them. This is called *overfitting* in the machine-learning literature and is the reverse of *underfitting*, where we have not learned enough. To overcome overfitting, we need a separate dataset other than training set, on which we will compare learning algorithms. This set is called *test set*. We also need a third set *validation set*, on which we tune the hyperparameters of the algorithm. For example, in k -nearest neighbor, k is a hyperparameter and training on different k s produce different training errors, and in order to finalize the learner and test in on the test set, we need a separate set to differentiate between different k s.

Another important factor in learning is the impact of chance. Maybe there were one or more mislabeled instances in the training/validation/test set, or there were some outliers in the given training/validation pair, or there was noise while obtaining the features on the training/validation/test set. Yet another important possibility is the factor of chance on the training of the algorithm. Many learning algorithms (i.e., neural networks, K -means clustering) have an iterative type of optimization, where initially we start with a random solution. The initial solution may have a well impact on the end result. So, whatever possible reason is, training and validation on a single training/validation set pair is not healthy. We need to run our learning algorithm(s) on multiple sets, so we need to resample the dataset to obtain multiple training and validation sets.

Last but not least, remember that, error is not the only one criterion on which we compare algorithms. There are also other criteria in real life, which may get our attention more than the error, such as training time or space complexity, testing time or space complexity, interpretability, and risks when errors are generalized to other loss functions [36].

3.6.1 Sensitivity analysis

The aim of sensitivity analysis is to find the parameter/a set of parameters which has/have the greatest impact on the output of the model/algorithm [37]. It provides important insight into the model, whereby the most effective parameter/set of parameters is/are determined. In other words, sensitivity analysis helps the experiment designer to understand the input–output relationship; determines how the uncertainty in parameters affects the actual output of the system; and guides experiment designer for future experiment designs. There are two important types of sensitivity analysis: (i) local and (ii) global.

3.6.1.1 Local sensitivity analysis

In local sensitivity analysis, one evaluates the change in the model output with respect to a change in a single parameter. Only small variations on one parameter are applied, and the effect of this simple variation is obtained via local sensitivity indices. Local sensitivity indices are usually calculated as the partial derivatives of the model output with respect to that local parameter.

The main limitation of the local sensitivity analysis is that it evaluates the effect of one parameter at a time, and therefore does not allow the evaluation of simultaneous effects of changes in multiple parameters, and does not allow calculations of interactions between multiple parameters.

3.6.1.2 Global sensitivity analysis

In global sensitivity analysis, all parameters of the system are changed at the same time over the whole parameter space, which enables us to calculate the respective contributions of each parameter and the interactions between those multiple parameters. The interactions between the model parameters identify the model output variance.

Well-known global sensitivity analysis methodologies are

- Weighted average of local sensitivity analysis, which calculates the local sensitivity analysis indices at different random values of the input parameters. The weighted average of the local indices is then used to calculate the global sensitivity.
- Partial rank correlation coefficient, which uses the Pearson correlation coefficients (between -1 and 1) to identify the important parameters.
- Sobol method, which is based on variance decomposition technique to obtain the corresponding contributions of the input parameters on the model output variance. Sobol's method also can identify the interactions on the input parameters on the overall output variance.

3.6.2 Resampling

In this section, we will talk about how to generate K training/validation set pairs. We also want to hold back as much as from overlapping the validation and training set pairs.

3.6.2.1 K -Fold cross-validation

In K -fold cross-validation, the aim is to generate K training/validation set pair, where training and validation sets on fold i do no overlap. First, we divide the dataset $\mathbf{X}$ into K parts as $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_K$. Then for each fold i , we use $\mathbf{X}_i$ as the validation set and $\mathbf{X} - \mathbf{X}_i$ as the training set. So, the training and validation sets are

$$\begin{aligned} \mathbf{T}_1 &= \mathbf{X}_2 \cup \mathbf{X}_3 \cup \dots \mathbf{X}_K & \mathbf{V}_1 &= \mathbf{X}_1 \\ \mathbf{T}_2 &= \mathbf{X}_1 \cup \mathbf{X}_3 \cup \dots \mathbf{X}_K & \mathbf{V}_2 &= \mathbf{X}_2 \\ &\dots & & \\ \mathbf{T}_K &= \mathbf{X}_1 \cup \mathbf{X}_2 \cup \dots \mathbf{X}_{K-1} & \mathbf{V}_K &= \mathbf{X}_K \end{aligned}$$

Possible values of K are 10 or 30. One extreme case of K -fold cross-validation is leave-one-out, where $K = N$ and each validation set has only one instance.

If we have more computation power, we can have multiple runs of K -fold cross-validation, such as 10×10 cross-validation [38] or 5×2 cross-validation [8,39].

3.6.2.2 Bootstrapping

If we have very small datasets, we do not insist on the non-overlap of training and validation sets. In bootstrapping, we generate K multiple training sets, where each training set contains N examples (like the original dataset). To get N examples, we draw examples with replacement. For the validation set, we use the original dataset. The drawback of bootstrapping is that the bootstrap samples overlap more than the cross-validation sample, hence they are more dependent.

3.6.3 Comparison of algorithms

In this section, we will talk about statistical tests comparing two classification algorithms, and test whether those two algorithms have the same error rate.

3.6.3.1 K -Fold cv paired t -test

In K -fold cv paired t -test, we assume that we sampled the training set with K -fold cross-validation and the difference of error rates of two algorithms on fold i are calculated as $e_i = f_i - s_i$, where f_i and s_i represent the error rates of the first and second algorithms on fold i , respectively.

The null hypothesis of the K -fold cv paired t -test is that the distribution of e_i has 0 mean. The test statistic:

$$\tau_K = \frac{\sqrt{K}m}{S} \tag{3.34}$$

is t -distributed with $K - 1$ degrees of freedom, where m and S are the mean and the variance of e_i 's. The test rejects the null hypothesis at significance level α , if τ_K is outside the interval $(-t_{\alpha/2, K-1}, t_{\alpha/2, K-1})$.

3.6.3.2 5×2 cv paired t -test

In 5×2 cv paired t -test [39], we assume that we sampled the training set with 5×2 -fold cross-validation and the difference of error rates of two algorithms on fold j of replication i are calculated as $e_{ij} = f_{ij} - s_{ij}$, where f_{ij} and s_{ij} represent the error rates of the first and second algorithms on fold j of replication i , respectively.

Under the null hypothesis that the two classifications have the same error rate, the test statistic

$$\tau_t = \frac{e_{11}}{\sqrt{\sum_{i=1}^5 s_i^2 / 5}} \quad (3.35)$$

is t -distributed with 5 degrees of freedom, where s_i^2 is the variance of the error differences on replication i . The test rejects the null hypothesis at significance level α , if τ_t is outside the interval $(-t_{\alpha/2,5}, t_{\alpha/2,5})$.

3.6.3.3 Combined 5×2 cv F -test

In combined 5×2 cv F -test [8], the assumptions are the same as 5×2 cv paired t -test.

Under the null hypothesis that two classifications have the same error rate, the test statistic:

$$\tau_F = \frac{\sum_{i=1}^5 \sum_{j=1}^2 e_{ij}^2}{2 \sum_{i=1}^5 s_i^2} \quad (3.36)$$

is F -distributed with 10 and 5 degrees of freedoms. The test rejects the null hypothesis at significance level α , if τ_F is greater than $F_{\alpha,10,5}$.

3.6.3.4 Combined 5×2 cv t -test

In combined 5×2 cv t -test [9], the assumptions are the same as 5×2 cv paired t -test.

Under the null hypothesis that two classifications have the same error rate, the test statistic:

$$\tau_C = \frac{\sum_{i=1}^5 \sum_{j=1}^2 e_{ij}}{\sqrt{\sum_{i=1}^5 \sum_{j=1}^2 e_{ij}^2 - 2 \sum_{i=1}^5 e_{i1} e_{i2}}} \quad (3.37)$$

is t -distributed with 5 degrees of freedom. The test rejects the null hypothesis at significance level α , if τ_C is outside the interval $(-t_{\alpha/2,5}, t_{\alpha/2,5})$.

References

- [1] Vapnik V. The Nature of Statistical Learning Theory. New York, NY: Springer Verlag; 1995.
- [2] Alpaydm E. Introduction to Machine Learning. Cambridge, MA: The MIT Press; 2010.

- [3] Bishop CM. Neural Networks for Pattern Recognition. Oxford, England: Oxford University Press; 1995.
- [4] Quinlan JR. C4.5: Programs for Machine Learning. San Mateo, CA: Morgan Kaufmann; 1993.
- [5] Pearl J. Causality: Models, Reasoning, and Inference. Cambridge, England: Cambridge University Press; 2000.
- [6] Jordan MI. An Introduction to Probabilistic Graphical Models. Cambridge, MT: The MIT Press; 2009.
- [7] Rabiner LR. A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition. *Proceedings of the IEEE*. 1989;77:257–286.
- [8] Alpaydın E. Combined 5×2 cv F Test for Comparing Supervised Classification Learning Classifiers. *Neural Computation*. 1999;11: 1975–1982.
- [9] Yıldız OT. Omnivariate Rule Induction Using a Novel Pairwise Statistical Test. *IEEE Transactions on Knowledge and Data Engineering*. 2013;25: 2105–2118.
- [10] Yıldız OT, and Alpaydın E. Omnivariate Decision Trees. *IEEE Transactions on Neural Networks*. 2001;12(6):1539–1546.
- [11] Fürnkranz J. Separate-and-Conquer Learning. *Artificial Intelligence Review*. 1999;13:3–54.
- [12] Fawcett T. An Introduction to ROC Analysis. *Pattern Recognition Letters*. 2006;27:861–874.
- [13] Cherkassky V, and Mulier F. Learning From Data. Hoboken, NJ: John Wiley and Sons; 1998.
- [14] Devjner PA, and Kittler J. Pattern Recognition: A Statistical Approach. New York, NY: Prentice-Hall; 1982.
- [15] Guyon I, and Elisseeff A. An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*. 2003;3:1157–1182.
- [16] Pudil P, Novovicova J, and Kittler J. Floating Search Methods in Feature Selection. *Pattern Recognition Letters*. 1994;15:1119–1125.
- [17] Jain AK, and Dubes RC. Algorithms for Clustering Data. New York, NY: Prentice-Hall; 1988.
- [18] Jain AK, Murthy MN, and Flynn PJ. Data Clustering: A Review. *ACM Computing Surveys*. 1999;31:264–323.
- [19] Dempster AP, Laird NM, and Rubin DB. Maximum Likelihood from Incomplete Data via the EM Algorithm. *Journal of Royal Statistical Society B*. 1977;39:1–38.
- [20] Redner RA, and Walker HF. Mixture Densities, Maximum Likelihood and the EM Algorithm. *SIAM Review*. 1984;26:195–239.
- [21] Wolpert DH. The Relationship between PCA, the Statistical Physics Framework, the Bayesian Framework, and the VC Framework. In: Wolpert DH, editor. *The Mathematics of Generalization*. Cambridge, MA: Addison Wesley; 1995. p. 117–214.
- [22] Aha DW, Kibler D, and Albert MK. Instance-Based Learning Algorithm. *Machine Learning*. 1991;6:37–66.

- [23] Aha DW. Special Issue on Lazy Learning. *Artificial Intelligence Review*. 1997;11(1–5):7–423.
- [24] Duda RO, Hart PE, and Stork DG. *Pattern Classification*. Hoboken, NJ: John Wiley and Sons; 2001.
- [25] Yıldız OT, and Alpaydın E. Ordering and Finding the Best of $K > 2$ Supervised Learning Algorithms. *IEEE Transactions on Pattern Analysis Machine Intelligence*. 2006;28(3):392–402.
- [26] Murthy SK. Automatic Construction of Decision Trees from Data: A Multi-Disciplinary Survey. *Data Mining and Knowledge Discovery*. 1998;2(4):345–389.
- [27] Yıldız OT, and Alpaydın E. Linear Discriminant Trees. In: 17th International Conference on Machine Learning. Morgan Kaufmann; 2000. p. 1175–1182.
- [28] Guo H, and Gelfand SB. Classification Trees With Neural Network Feature Extraction. *IEEE Transactions on Neural Networks*. 1992;3:923–933.
- [29] Breiman L, Friedman JH, Olshen RA, *et al.* *Classification and Regression Trees*. Hoboken, NJ: John Wiley and Sons; 1984.
- [30] Quinlan JR. Induction of Decision Trees. *Machine Learning*. 1986;1:81–106.
- [31] Vapnik V. *Statistical Learning Theory*. New York, NY: Wiley; 1998.
- [32] Schölkopf B, Tsuda K, and Vert JP, editors. *Kernel Methods in Computational Biology*. Cambridge, MA: The MIT Press; 2004.
- [33] Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. New York, NY: Spartan; 1962.
- [34] Minsky ML, and Papert SA. *Perceptrons*. Cambridge, MA: MIT Press; 1969.
- [35] Hopfield JJ. Neural Networks and Physical Systems with Emergent Collective Computational Abilities. *Proceedings of the National Academy of Sciences*. 1982;79:2554–2558.
- [36] Turney PD. Types of Cost in Inductive Concept Learning. In: Workshop on Cost-Sensitive Learning in 17th International Conference on Machine Learning. Stanford University, CA, USA; 2000. p. 15–21.
- [37] Saltelli A, Chan K, and Scott EM. *Sensitivity Analysis: Wiley Series in Probability and Statistics*. New York, NY: Wiley; 2000.
- [38] Bouckaert RR, and Frank E. Evaluating the Replicability of Significance Tests for Comparing Learning Algorithms. In: *Advances in Knowledge Discovery and Data Mining, LNCS*. vol. 3056. Sydney, Australia: Springer; 2004. p. 3–12.
- [39] Dietterich TG. Approximate Statistical Tests for Comparing Supervised Classification Learning Classifiers. *Neural Computation*. 1998;10:1895–1923.